АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

Полный конспект-шпаргалка к командному коллоквиуму (1 семестр)

Университет ИТМО • Факультет ИТМО • Группа М3102 • 2026

📌 Структура подготовки к коллоквиуму

Для каждой темы в билете обязательно проверяются **5 ключевых аспектов**:

1. **Оценка сложности**: по времени (лучший, средний, худший случаи) и по дополнительной памяти.
2. **Структура данных**: на чём реализуется логически и физически.
3. **Ограничения и свойства**: устойчивость, in-place, требования к ключам, переполнения типов, кэш-локальность.
4. **Применимость**: когда алгоритм оптимален, где применяется на практике и в стандартных библиотеках.
5. **Логика работы**: инвариант, пошаговые действия, математические доказательства ключевых оценок.

Оглавление тем коллоквиума

РАЗДЕЛ 0. СВОДНЫЕ МАТРИЦЫ ДЛЯ БЫСТРОГО ПОВТОРЕНИЯ 2
0.1. Сравнительная матрица алгоритмов сортировки 2
0.2. Сравнительная матрица базовых структур данных 2
0.3. Сравнительная матрица динамического программирования 3
РАЗДЕЛ 1. БАЗОВЫЕ ТЕМЫ КОЛЛОКВИУМА (ЧАСТЬ 1) 4
ТЕМА 1. Оценка сложности алгоритма по времени 4
ТЕМА 2. Оценка сложности алгоритма по памяти 4
ТЕМА 3. Сортировка вставками (Insertion Sort) 5
ТЕМА 4. Сортировка слиянием (Merge Sort) 5
ТЕМА 5. Быстрая сортировка (Quick Sort) 6
ТЕМА 6. Сортировка подсчетом (Counting Sort) 6
ТЕМА 7. Цифровая сортировка (Radix Sort) 7
ТЕМА 8. Стек (Stack) 7
ТЕМА 9. Очередь (Queue) 8
ТЕМА 10. Односвязный список (Singly Linked List) 8
РАЗДЕЛ 2. БАЗОВЫЕ ТЕМЫ КОЛЛОКВИУМА (ЧАСТЬ 2) 10
ТЕМА 11. Двусвязный список (Doubly Linked List) 10
ТЕМА 12. Циклический список (Circular Linked List) 10
ТЕМА 13. Стек на списках (Stack on Linked List) 10
ТЕМА 14. Очередь на списках (Queue on Linked List) 11
ТЕМА 15. Пирамида (Куча / Binary Heap) 11
ТЕМА 16. Построение пирамиды за $O(n)$ (Build Heap) 12
ТЕМА 17. Пирамидальная сортировка (HeapSort) 12
ТЕМА 18. Удаление элемента из пирамиды 13
ТЕМА 19. Очередь с приоритетом (Priority Queue) 13
ТЕМА 20. Динамическое программирование (Основы + 3 задачи) 13
РАЗДЕЛ 3. ПРОДВИНУТЫЕ ТЕМЫ СО ЗВЁЗДОЧКОЙ (*) 15
ТЕМА *1. Квикселект (Quickselect и алгоритм BFPRT) 15
ТЕМА *2. Мастер-теорема как способ доказательства 15
ТЕМА *3. Дин прог: битовые маски, профиль, редакционное расстояние 16
ТЕМА *4. Дек и Очередь с поддержкой минимума 16
ТЕМА *5. Бинпоиск: lower_bound, по ответу, вещественный 17

РАЗДЕЛ 0. СВОДНЫЕ МАТРИЦЫ ДЛЯ БЫСТРОГО ПОВТОРЕНИЯ

0.1. Сравнительная матрица алгоритмов сортировки

Алгоритм	Лучшее	Среднее	Худшее	Память	Устойч.	In-place	Ограничения / Ключ
Сортировка вставками (Insertion)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Да	Да	Медленно на больших n
Сортировка слиянием (Merge)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Да	Нет	Требует $O(n)$ памяти
Быстрая сортировка (Quick)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Нет	Да	Худший случай $O(n^2)$
Сортировка подсчетом (Counting)	$O(n+K)$	$O(n+K)$	$O(n+K)$	$O(n+K)$	Да*	Нет	Ключи целые, $K = O(n)$
Цифровая сортировка (Radix LSD)	$O(d(n+B))$	$O(d(n+B))$	$O(d(n+B))$	$O(n+B)$	Да	Нет	Разбиение на разряды
Пирамидальная (HeapSort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Нет	Да	Плохая кэш-локальность
Сортировка выбором (Selection)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Нет	Да	Всегда делает $O(n^2)$ сравн.
Сортировка пузырьком (Bubble)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Да	Да	Медленная, много обменов

* Примечание: Counting sort устойчив в варианте с префиксными суммами при обратном проходе (справа налево). Память QuickSort $O(\log n)$ достигается устранением хвостовой рекурсии для большего подмассива.

0.2. Сравнительная матрица базовых структур данных

Структура данных	Индекс [i]	Push Front	Push Back	Pop Front	Pop Back	Оверхед памяти
Динамический массив (Vector)	$O(1)$	$O(n)$	$O(1)^{\text{am}}$	$O(n)$	$O(1)$	Запас емкости (до $2\times$)
Односвязный список (SList)	$O(n)$	$O(1)$	$O(1)^{*}$	$O(1)$	$O(n)$	1 указатель на узел (8B)
Двусвязный список (DList)	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	2 указателя на узел (16B)
Стек (динамический массив)	—	—	$O(1)^{\text{am}}$	—	$O(1)$	Минимальный (подряд в RAM)
Стек (на связном списке)	—	$O(1)$	—	$O(1)$	—	8 байт указатель + malloc
Очередь (кольцевой буфер)	—	—	$O(1)^{\text{am}}$	$O(1)$	—	Минимальный в кольце
Очередь (на связном списке)	—	—	$O(1)$	$O(1)$	—	Указатели + malloc
Дек (кольцевой буфер / чанки)	$O(1)$	$O(1)^{\text{am}}$	$O(1)^{\text{am}}$	$O(1)$	$O(1)$	Массив указателей на чанки
Двоичная куча (Binary Heap)	$O(1)^{\text{root}}$	—	$O(\log n)$	—	$O(\log n)$	$O(1)$ в непрерывном массиве

* Добавление в конец односвязного списка за $O(1)$ возможно при хранении указателя tail; удаление конца требует $O(n)$, так как нужно обновить предпоследний узел.

0.3. Сравнительная матрица динамического программирования

Задача ДП	Состояние dp	Формула перехода	Время	Память
Кузнечик	$\mathrm{dp}[i]$: макс. монет до кочки i	$\mathrm{dp}[i] = c_i + \max_{1 \le j \le k} \mathrm{dp}[i-j]$	$O(nk) \to O(n)^*$	$O(n) \to O(k)$
Рюкзак 0-1	$\mathrm{dp}[i][w]$: макс. стоимость первых i предм. с весом $\le w$	$\mathrm{dp}[i][w] = \max(\mathrm{dp}[i-1][w], \mathrm{dp}[i-1][w-w_i] + c_i)$	$O(nW)$	$O(nW) \to O(W)$
НВП (LIS) $O(n^2)$	$\mathrm{dp}[i]$: длина НВП, оканч. на $A[i]$	$\mathrm{dp}[i] = 1 + \max_{j<i, A[j]<A[i]} \mathrm{dp}[j]$	$O(n^2)$	$O(n)$
НВП (LIS) $O(n \log n)$	$d[\mathrm{len}]$: мин. конец НВП длины len	Бинарный поиск $\mathrm{lower_bound}$ в массиве d	$O(n \log n)$	$O(n)$
Редакционное расст.	$\mathrm{dp}[i][j]$: Левенштейн для $S_1[0..i-1]$ и $S_2[0..j-1]$	$\mathrm{dp}[i][j] = \min(\mathrm{del}, \mathrm{ins}, \frac{\mathrm{sub}}{\mathrm{match}})$	$O(nm)$	$O(nm) \to O(\min(n,m))$
Динамика по профилю	$\mathrm{dp}[i][\mathrm{mask}]$: число способов замостить сетку	$\mathrm{dp}[i][\mathrm{mask}] = \sum_{\mathrm{prev}} \mathrm{dp}[i-1][\mathrm{prev}] \cdot \mathrm{can}(\mathrm{prev}, \mathrm{mask})$	$O(M 2^{2N}) \to O(NM2^N)$	$O(2^N)$

* Ускорение кузнечика до $O(n)$ достигается с помощью очереди с максимумом / монотонного дека.

РАЗДЕЛ 1. БАЗОВЫЕ ТЕМЫ КОЛЛОКВИУМА (ЧАСТЬ 1)

ТЕМА 1. Оценка сложности алгоритма по времени

Лучшее время	Среднее время	Худшее время	Доп. память
$O(1)$ **(мин. шагов)**	**Мат. ожидание** $E[T(n)]$	**Макс. число шагов**	**Не применимо**

1. На какой структуре данных реализуется: Математическая абстракция над моделью вычислений (RAM / word RAM).

2. Логика работы и теоретическая база:
- **Функция трудоёмкости** $T(n)$ — точное число элементарных (атомарных) операций машины при размере входа n.
- **Модель RAM (Random Access Machine)**: память состоит из бесконечной ленты ячеек с произвольным доступом за $O(1)$.
- **Модель word RAM**: размер ячейки памяти (машинного слова) составляет $w \geq \log_2 n$ бит. Базовые операции (сложение, умножение, побитовые сдвиги, чтение/запись ячейки) выполняются за $O(1)$ машинный такт.
- **Асимптотические обозначения**:
  - $O(g(n))$ **(верхняя граница)**: $\exists c > 0, n_0 : \forall n \geq n_0 \quad 0 \leq f(n) \leq c \cdot g(n)$.
  - $\Omega(g(n))$ **(нижняя граница)**: $\exists c > 0, n_0 : \forall n \geq n_0 \quad f(n) \geq c \cdot g(n) \geq 0$.
  - $\Theta(g(n))$ **(точная асимптотика)**: $f(n) \in O(g(n))$ и $f(n) \in \Omega(g(n))$. Существуют $c_1, c_2 > 0 : c_1 g(n) \leq f(n) \leq c_2 g(n)$.
  - $o(g(n))$ **(строго меньше)**: $\forall c > 0 \quad \exists n_0 : \forall n \geq n_0 \quad 0 \leq f(n) < c \cdot g(n) \Leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.
  - $\omega(g(n))$ **(строго больше)**: $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$.

3. Ограничения и свойства:
- Константы и младшие степени отбрасываются: $5n^2 + 100n + 42 \in \Theta(n^2)$.
- Не путать худший случай алгоритма с O-нотацией! У худшего случая алгоритма может быть оценка как Θ, так и O, и Ω.

4. Применимость: Сравнение алгоритмов независимо от тактовой частоты процессора, компилятора и операционной системы.

> 💡 **Мнемоника для запоминания**
> O — крыша (не выше), Ω — пол (не ниже), Θ — коридор между полом и крышей.

—

ТЕМА 2. Оценка сложности алгоритма по памяти

Лучшее время	Среднее время	Худшее время	Доп. память
In-place $O(1)$	**Среднее исп. памяти**	**Пиковый объём памяти**	**Auxiliary Space**

1. На какой структуре данных реализуется: Оперативная память процесса: стек вызовов (Call Stack) и динамическая куча (Heap).

2. Логика работы и теоретическая база:
- **Входная/выходная память** (Input/Output space) — память для хранения самого входа и ответа (например, входной массив длины n уже занимает $O(n)$ памяти).
- **Дополнительная рабочая память (Auxiliary Space)** — память, выделяемая **сверх** входа для внутренних нужд алгоритма (переменные, буферы, стек рекурсии). Именно её называют «памятью алгоритма».
- **Стек вызовов рекурсии**: каждый рекурсивный вызов создает в стеке **стековый кадр (stack frame)** с аргументами, адресом возврата и локальными переменными. Если максимальная глубина дерева рекурсии равна h, то расход стековой памяти составляет $O(h)$.
- **In-place (на месте)**: алгоритм называется in-place, если ему требуется $O(1)$ (или $O(\log n)$ для стека QuickSort) дополнительной памяти.

3. Ограничения и типичные ловушки:
- Рекурсия глубины n при размере кадра 32 байта при $n = 10^6$ приведет к **Stack Overflow** (переполнению стека, размер которого обычно 8 МБ).
- Динамическое выделение (`new` / `malloc`) сопровождается скрытым оверхедом заголовка блока (8–16 байт на каждый блок в куче) и фрагментацией.

> 💡 **Мнемоника для запоминания**
> Глубина стека рекурсии = высота дерева вызовов! Для сбалансированного деления — $O(\log n)$, для вырожденного — $O(n)$.

—

TEMA 3. Сортировка вставками (Insertion Sort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(n)$ **(отсортирован)**	$O(n^2)$	$O(n^2)$ **(обратный)**	$O(1)$ **(In-place)**

1. На какой структуре данных реализуется: Массив с произвольным доступом или связный список.

2. Ограничения и свойства:
- **Устойчивость (Stable)**: ДА (равные элементы не меняют относительный порядок, так как строгое сравнение $A[j] > x$ останавливает сдвиг).
- **In-place**: ДА (требует ровно 1 переменную x под текущий элемент).
- **Адаптивность**: ДА (время линейно зависит от степени отсортированности входа).

3. Применимость:
- Массивы малого размера ($n \leq 16...32$). Используется как базовый случай в промышленных гибридных сортировках: **Timsort** (Python, Java) и **Introsort** (`std::sort` в C++).
- Почти отсортированные массивы (с малым числом нарушений порядка).
- Онлайн-алгоритм: новые данные вставляются в уже отсортированный поток на лету.

4. Логика работы и инвариант:
- **Инвариант внешнего цикла:** перед началом шага i подмассив $A[0..i-1]$ уже упорядочен по возрастанию.
- **Шаги**:
  1. Берём элемент $x = A[i]$.
  2. Просматриваем элементы слева от i ($j = i-1$ вниз до 0): пока $A[j] > x$, сдвигаем $A[j]$ вправо: $A[j+1] = A[j]$.
  3. Как только $A[j] \leq x$ или дошли до начала, вставляем x на освободившуюся позицию $A[j+1] = x$.
- **Связь с инверсиями**: число сравнений и сдвигов в точности равно $I + O(n)$, где I — число пар инверсий ($i < j$ и $A[i] > A[j]$). В худшем случае обратного массива $I = \frac{n(n-1)}{2} \in \Theta(n^2)$.

> 💡 **Мнемоника для запоминания**
> Раскладывание карт в руке: берем очередную карту и просовываем её на правильное место среди уже разложенных.

—

TEMA 4. Сортировка слиянием (Merge Sort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ **(массив)** / $O(1)$ **(список)**

1. На какой структуре данных реализуется: Массив или связный список.

2. Ограничения и свойства:
- **Устойчивость (Stable)**: ДА (при слиянии при равенстве элементов $L[i] == R[j]$ всегда первее берется элемент из левой половины L).
- **In-place**: НЕТ на массивах (требует вспомогательный буфер $O(n)$ для слияния), но ДА на связных списках (переставляются только указатели).
- Не зависит от начального порядка: работает строго $\Theta(n \log n)$ в любом случае.

3. Применимость:
- Сортировка связных списков (`std::list::sort`).
- Внешняя сортировка (External sorting): сортировка терабайтных файлов на жестких дисках, не помещающихся в RAM (k-way merge).
- Задачи подсчета числа инверсий в массиве за $O(n \log n)$.

4. Логика работы:
- Парадигма: **Разделяй и властвуй (Divide and Conquer)**.
- **Рекуррента**: $T(n) = 2T(\frac{n}{2}) + \Theta(n) = \Theta(n \log n)$ по мастер-теореме.

- **Алгоритм Merge**: два указателя i и j бегут по двум отсортированным подмассивам. На каждом шаге меньший элемент записывается в выходной буфер, соответствующий указатель сдвигается. После исчерпания одного подмассива остаток второго дописывается в конец.
- **Итеративная версия (Bottom-up)**: без рекурсии сливает блоки длины $1 \to 2 \to 4 \to 8 \ldots$.
- **Подсчёт инверсий**: когда при слиянии элемент берется из правого массива $R[j]$, он образует инверсию со всеми оставшимися элементами левого массива! К счетчику инверсий прибавляется $(\text{len}_L - i)$.

> 💡 **Мнемоника для запоминания**
> Две стопки сданных экзаменационных работ: смотрим на верхние работы в обеих стопках, меньшую кладем в итоговую стопку.

—

ТЕМА 5. Быстрая сортировка (Quick Sort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(n \log n)$	$O(n \log n)$	$O(n^2)$ **(плохой pivot)**	$O(\log n)$ **(стек вызовов)**

1. На какой структуре данных реализуется: Массив с произвольным доступом.

2. Ограничения и свойства:
- **Устойчивость (Stable)**: НЕТ (дальние обмены элементов через опорный нарушают относительный порядок равных ключей).
- **In-place**: ДА (сортирует внутри исходного массива, память только на стек рекурсии).
- **Худший случай**: $O(n^2)$ возникает, когда на каждом шаге выбирается экстремальный элемент (минимум или максимум), и массив делится на размеры 0 и $n-1$.

3. Применимость:
- Самая быстрая сортировка общего назначения на практике благодаря прекрасной пространственной локальности данных (последовательные обращения к памяти загружают кэш-линии процессора). Основа функции `std::sort` (Introsort переключается на HeapSort при превышении глубины рекурсии $2 \log n$).

4. Логика работы:
1. **Выбор pivot (опорного)**: случайный элемент, медиана трех (начало, середина, конец).
2. **Разбиение (Partition)**:
   - **Схема Ломуто**: простой указатель i, сдвигающий элементы $< \text{pivot}$ влево. Делает много свопов.
   - **Схема Хоара**: два указателя i и j идут навстречу друг другу. Делает в 3 раза меньше свопов и эффективнее на массивах с дубликатами.
3. **Оптимизация памяти стека рекурсии до $O(\log n)$**: рекурсивный вызов делается **только для меньшей** половины, а для большей используется цикл (устранение хвостовой рекурсии).

> 💡 **Мнемоника для запоминания**
> Быстрая сортировка — это «сначала раздели по порогу, потом рекурсивно отсортируй половинки».

—

ТЕМА 6. Сортировка подсчетом (Counting Sort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(n + K)$	$O(n + K)$	$O(n + K)$	$O(n + K)$ **(массив вывода)**

1. На какой структуре данных реализуется: Массив частот и массив префиксных сумм.

2. Ограничения и свойства:
- **Требование к ключам**: целые неотрицательные числа в компактном диапазоне $[0, K-1]$.
- Если $K \gg n$ (например, 10 чисел со значениями до 10^9), алгоритм крайне неэффективен ($O(10^9)$ времени и памяти).
- **Устойчивость**: ДА (при использовании префиксных сумм и обходе массива справа налево).
- **In-place**: НЕТ (требует буфер $O(K)$ для подсчета и $O(n)$ для вывода).

3. Применимость:
- Сортировка данных с малым алфавитом: баллы ЕГЭ (0..100), возраст людей (0..150), символы ASCII (0..255).
- Фундаментальный строительный блок для поразрядной сортировки (Radix Sort).

4. Логика работы (устойчивая версия):
1. Создаем массив $C[0..K-1] = 0$.

2. Подсчитываем частоты: для каждого $x \in A$ делаем $C[x] + +$.
3. Превращаем в префиксные суммы: $C[i] = C[i] + C[i-1]$. Теперь $C[x]$ показывает, сколько элементов $\leq x$ должно стоять в отсортированном массиве (то есть индекс конца диапазона для x).
4. Формируем выходной массив B: идем по исходному массиву A **справа налево** от $n-1$ до 0:
   - Берём `val = A[i]`;
   - Позиция `pos = C[val] - 1`;
   - Ставим `B[pos] = val`;
   - Декрементируем счетчик: `C[val]--`.

- **Почему справа налево?** Элемент, стоявший правее, займет наибольшую доступную ячейку для своего значения, сохранив относительный порядок дубликатов!

> 💡 **Мнемоника для запоминания**
> Гардероб с номерками: считаем, сколько курток на каждой вешалке, затем раскладываем по местам.

—

TEMA 7. Цифровая сортировка (Radix Sort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(d(n+B))$	$O(d(n+B))$	$O(d(n+B))$	$O(n+B)$

1. На какой структуре данных реализуется: Массивы счетчиков и буферы вывода.

2. Ограничения и свойства:
- Ключи должны разбиваться на d разрядов фиксированного размера в системе счисления по основанию B.
- **Устойчивость**: ДА (для варианта LSD — Least Significant Digit).
- **In-place**: НЕТ.

3. Применимость:
- Сортировка 32-битных и 64-битных целых чисел (при $B = 256 = 2^8$ число делится на 4 байта, $d = 4$, сортировка выполняется за 4 прохода Counting Sort).
- Строки фиксированной длины (UUID, хэши SHA-256, телефонные номера, почтовые индексы).

4. Логика работы:
- **LSD (от младших разрядов к старшим)**:
  1. Для разряда k от 0 до $d-1$:
  2. Запускаем устойчивую сортировку (Counting Sort) массива по k-му разряду.
  3. Поскольку подсчет устойчив, элементы, равные по текущему разряду, сохраняют взаимный порядок, полученный на предыдущих (младших) разрядах.
  4. После прохода по старшему разряду массив полностью отсортирован!
- **MSD (от старших к младшим)**: разбивает массив на корзины по старшему разряду и затем рекурсивно сортирует каждую корзину отдельно (аналогично Trie).

> 💡 **Мнемоника для запоминания**
> LSD сортирует числа как даты: сначала по дням, потом по месяцам, потом по годам. Устойчивость гарантирует, что внутри одного года месяцы и дни не перемешаются!

—

TEMA 8. Стек (Stack)

Лучшее время	Среднее время	Худшее время	Доп. память
Push / Pop: $O(1)$	**Top / Peek:** $O(1)$	**Поиск:** $O(n)$	**Память:** $O(n)$

1. На какой структуре данных реализуется:
- Динамический массив (вектор с авто-расширением) — амортизированное $O(1)$ на добавление.
- Односвязный список с добавлением и удалением из головы — строгое $O(1)$ в худшем случае.

2. Ограничения и свойства:
- Принцип: **LIFO (Last In, First Out)** — последним пришел, первым ушел.
- Произвольный доступ по индексу запрещен (доступен только верхний элемент `top`).

3. Применимость:
- Проверка правильности скобочных последовательностей (баланс скобок).
- Вычисление выражений в обратной польской нотации (RPN / постфиксная запись).

- Стек вызовов функций (Call Stack) в JVM, CLR и бинарном коде.
- Отмена операций (Undo/Redo) в текстовых редакторах.
- Реализация нерекурсивного обхода графа в глубину (DFS).

4. Логика работы:
- `push(x)`: кладем элемент на вершину стека (`a[++top] = x`).
- `pop()`: удаляем элемент с вершины (`return a[top--]`).
- `top()` / `peek()`: возвращаем значение на вершине без удаления.
- `isEmpty()`: проверка `top == -1`.

> 💡 **Мнемоника для запоминания**
> Стопка тарелок: чистую тарелку кладут наверх стопки, берут для еды тоже всегда самую верхнюю.

—

ТЕМА 9. Очередь (Queue)

Лучшее время	Среднее время	Худшее время	Доп. память
Enqueue: $O(1)$	**Dequeue:** $O(1)$	**Front:** $O(1)$	**Память:** $O(n)$

1. На какой структуре данных реализуется:
- Кольцевой буфер (Ring Buffer) на массиве фиксированной или динамической емкости.
- Односвязный список с двумя указателями (`head` и `tail`).

2. Ограничения и свойства:
- Принцип: **FIFO (First In, First Out)** — первым пришел, первым ушел.
- Произвольный доступ запрещен (добавление только в хвост, извлечение только из головы).

3. Применимость:
- Алгоритм поиска в ширину (BFS) в графах и на деревьях.
- Системы буферизации пакетов в сети и очередей сообщений (RabbitMQ, Kafka).
- Планировщики задач ОС (очередь процессов на исполнение CPU).

4. Логика работы (Кольцевой буфер на массиве длины N):
- Хранятся два индекса: `head` (голова, откуда извлекаем) и `tail` (куда вставляем следующий), а также текущий размер `sz`.
- `enqueue(x)`: $A[\text{tail}] = x$; $\text{tail} = (\text{tail} + 1) \bmod N$; `sz++`.
- `dequeue()`: `val = A[head]`; $\text{head} = (\text{head} + 1) \bmod N$; `sz--`; return `val`.
- Если `sz == N` — буфер полон. При динамическом расширении массив удваивается, а элементы «распрямляются» от 0 до `sz - 1`.

> 💡 **Мнемоника для запоминания**
> Очередь в кассу магазина: кто первый встал, того первого и обслужили. Новые покупатели подходят строго в хвост.

—

ТЕМА 10. Односвязный список (Singly Linked List)

Лучшее время	Среднее время	Худшее время	Доп. память
Вставка в голову: $O(1)$	**Удаление головы:** $O(1)$	**Поиск / Индекс:** $O(n)$	**8 байт/узел доп. памяти**

1. На какой структуре данных реализуется: Динамически выделяемые в куче узлы (`Node`), связанные однонаправленными указателями.

2. Ограничения и свойства:
- Узел хранит: полезную нагрузку (`value`) и один указатель `next`.
- Обход возможен **только в одном направлении** (вперед).
- Вставка и удаление **после** заданного узла — $O(1)$.
- Удаление **хвостового** узла — $O(n)$, так как невозможно за $O(1)$ узнать предпоследний узел.
- **Плохая локальность данных в кэше CPU**: узлы разбросаны по памяти, что приводит к постоянным промахам кэша (L1/L2 cache misses) по сравнению с непрерывным массивом.

3. Применимость:
- Списки с частой вставкой/удалением в начало без накладных расходов на переаллокацию.
- Разрешение коллизий в хеш-таблицах методом цепочек (Separate Chaining).

- Списки смежности графа.

4. Логика работы основных операций:

- `push_front(x)`: `Node* p = new Node(x); p->next = head; head = p;` ($O(1)$).
- `insert_after(node, x)`: `Node* p = new Node(x); p->next = node->next; node->next = p;` ($O(1)$).
- `delete_after(node)`: `Node* tmp = node->next; node->next = tmp->next; delete tmp;` ($O(1)$).

> 💡 **Мнемоника для запоминания**
> Поезд из вагончиков: каждый вагончик сцеплен только со следующим за ним. Чтобы добраться до 10-го вагона, нужно пройти через первые 9.

РАЗДЕЛ 2. БАЗОВЫЕ ТЕМЫ КОЛЛОКВИУМА (ЧАСТЬ 2)

ТЕМА 11. Двусвязный список (Doubly Linked List)

Лучшее время	Среднее время	Худшее время	Доп. память
Вставка по указателю: $O(1)$	**Удаление по указателю:** $O(1)$	**Доступ по индексу:** $O(n)$	**16 байт/узел доп. памяти**

1. На какой структуре данных реализуется: Динамические узлы с двумя указателями (`prev` и `next`).

2. Ограничения и свойства:
- Удвоенный расход памяти на ссылки (16 байт на x64-системах только на указатели для каждого элемента).
- Требует аккуратности при обновлении 4 указателей при вставке и 2 при удалении во избежание утечек и повисших ссылок.

3. Применимость:
- **LRU-кэш** (Least Recently Used Cache): двусвязный список хранит порядок использования страниц/ключей, а хеш-таблица обеспечивает поиск узла за $O(1)$. При обращении узел вырезается за $O(1)$ и переносится в голову.
- История переходов в браузере (кнопки «Вперед» / «Назад»), плейлисты медиаплееров.
- Реализация контейнера `std::list` в C++.

4. Логика работы:
- **Удаление заданного узла P за $O(1)$**: в отличие от односвязного списка, не нужно искать предшественника!

```
P->prev->next = P->next;
P->next->prev = P->prev;
delete P;
```

- **Вставка нового узла перед P**: перенаправляются связи `prev` и `next` нового узла и его соседей.

> 💡 **Мнемоника для запоминания**
> Хоровод, где каждый держится за руки с обоими соседями. Если кто-то выходит из круга, соседи просто берутся за руки напрямую.

—

ТЕМА 12. Циклический список (Circular Linked List)

Лучшее время	Среднее время	Худшее время	Доп. память
Переход по кольцу: $O(1)$	**Вставка / удаление:** $O(1)$	**Полный обход:** $O(n)$	**8-16 байт/узел**

1. На какой структуре данных реализуется: Односвязный или двусвязный список, замкнутый в кольцо (`tail->next = head`).

2. Ограничения и свойства:
- Нет указателя `nullptr` в конце списка. Опасность вечного цикла `while(curr)` без правильного условия останова (`curr != head`).
- Техника **фиктивного сторожевого элемента (Sentinel / Dummy Node)**: в кольцо добавляется пустой узел-страж. Это устраняет проверки на пустоту списка и специальные случаи для головы и хвоста.

3. Применимость:
- Алгоритм планирования задач процессора **Round-Robin** (выделение кванта времени каждому процессу по кругу).
- Передача хода в многопользовательских настольных играх по часовой стрелке.
- Задача Иосифа Флавия (считалочка с выбыванием каждого k-го участника).

> 💡 **Мнемоника для запоминания**
> Карусель: после последней лошадки снова идет первая, конца нет.

—

ТЕМА 13. Стек на списках (Stack on Linked List)

Лучшее время	Среднее время	Худшее время	Доп. память
Push: строго $O(1)$	**Pop: строго** $O(1)$	**Top: строго** $O(1)$	**Оверхед памяти: ВЫСОКИЙ**

1. На какой структуре данных реализуется: Односвязный список с операциями на голове списка (`head`).

2. Сравнение со стеком на динамическом массиве:

Критерий	Стек на связном списке	Стек на динамическом массиве
Гарантия времени	Строго $O(1)$ в худшем случае	Амортизированное $O(1)$ (бывают паузы $O(n)$)
Расход памяти	Очень большой (8B link + 16B malloc)	Минимальный (только данные + резерв емкости)
Кэш процессора	Плохо (разброс в куче, cache miss)	Отлично (непрерывная область памяти, L1/L2 prefetch)
Перевыделение памяти	Отсутствует	Требуется realloc с копированием массива

3. Применимость: Системы жесткого реального времени (Hard Real-Time), где задержки на перевыделение памяти недопустимы. В остальных 99% задач динамический массив предпочтительнее.

—

ТЕМА 14. Очередь на списках (Queue on Linked List)

Лучшее время	Среднее время	Худшее время	Доп. память
Enqueue: строго $O(1)$	**Dequeue: строго $O(1)$**	**Front: строго $O(1)$**	**Оверхед памяти: ВЫСОКИЙ**

1. На какой структуре данных реализуется: Односвязный список с двумя указателями: `head` (извлечение) и `tail` (вставка).

2. Логика работы:
- `enqueue(x)`: создается `Node* p = new Node(x); tail->next = p; tail = p;`. Если очередь была пуста, `head = tail = p;`.
- `dequeue()`: запоминается `val = head->value; Node* tmp = head; head = head->next; delete tmp;`. Если `head == nullptr`, то `tail = nullptr`.
- **Сравнение с циклическим массивом**: не требует предварительного задания размера буфера, но тратит много памяти на указатели и вызовы `new`/`delete`.

—

ТЕМА 15. Пирамида (Куча / Binary Heap)

Лучшее время	Среднее время	Худшее время	Доп. память
Нахождение экстремума: $O(1)$	**SiftUp: $O(\log n)$**	**SiftDown: $O(\log n)$**	**Память: $O(1)$ in-place**

1. На какой структуре данных реализуется:
- **Логически**: Завершенное (почти полное) бинарное дерево (Complete Binary Tree), у которого заполнены все уровни, кроме, возможно, последнего, который заполняется слева направо.
- **Физически**: Обычный одномерный массив без указателей.

2. Индексация в массиве (0-based):
- Корень: индекс 0.
- Левый ребенок узла i: $2i + 1$.
- Правый ребенок узла i: $2i + 2$.
- Родитель узла i: $\lfloor \frac{i-1}{2} \rfloor$.

3. Основное свойство кучи (Heap Invariant):
- **Max-Heap**: значение в родителе $\geq$ значений обоих детей ($A\left[\lfloor \frac{i-1}{2} \rfloor\right] \geq A[i]$). Максимум всегда в корне $A[0]$.
- **Min-Heap**: значение в родителе $\leq$ значений обоих детей. Минимум всегда в корне $A[0]$.

4. Восстановление свойств кучи:
- `siftUp(i)` (**просеивание вверх**): пока $i > 0$ и $A[i] > A[\text{parent}(i)]$, меняем элемент с родителем и переходим к $i = \text{parent}(i)$. Высота дерева $h = \lfloor \log_2 n \rfloor \rightarrow$ время $O(\log n)$.
- `siftDown(i)` (**просеивание вниз**): сравниваем узел i с его детьми. Находим **наибольшего** из детей. Если ребенок больше $A[i]$, меняем местами и продолжаем просеивать вниз. Время $O(\log n)$.

> 💡 **Мнемоника для запоминания**
> Армейская иерархия: начальник всегда старше подчиненных по званию. Если рядовой отличился, его повышают вверх (`siftUp`). Если командир ошибся, его понижают вниз (`siftDown`).

—

ТЕМА 16. Построение пирамиды за $O(n)$ (Build Heap)

Лучшее время	Среднее время	Худшее время	Доп. память
Построение: $O(n)$	**Наивное (вставками):** $O(n \log n)$	**Худший случай:** $O(n)$	**Память:** $O(1)$ **in-place**

1. Алгоритм построения кучи «снизу вверх» (Bottom-Up): Берём массив произвольных n элементов и вызываем `siftDown(i)` для всех нелистовых вершин, двигаясь в обратном порядке:

```
for i in range(n // 2 - 1, -1, -1):
    sift_down(i)
```

Листья (индексы от $\lfloor \frac{n}{2} \rfloor$ до $n-1$) просеивать не нужно, так как поддерево из одного листа уже является корректной кучей.

> 📐 **Доказательство асимптотики O(n)**
> Высота узла h — это расстояние от узла до самого глубокого листа поддерева.
> • На высоте h находится не более $\lceil \frac{n}{2^{h+1}} \rceil$ вершин.
> • Стоимость `siftDown` для вершины на высоте h пропорциональна h.
>
> Суммарное число операций:
>
> $$T(n) = \sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) \leq cn \sum_{h=1}^{\infty} \frac{h}{2^{h+1}} = \frac{cn}{2} \sum_{h=1}^{\infty} \frac{h}{2^h}$$
>
> Сумма бесконечного ряда $S = \sum_{h=1}^{\infty} \frac{h}{2^h} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \ldots = 2$. Подставляя значение суммы ряда:
>
> $$T(n) \leq \frac{cn}{2} \cdot 2 = cn = O(n).$$

> ⚠️ **Ловушка на коллоквиуме**
> Почему построение через n вызовов `siftUp` (или последовательных вставок `insert`) работает за $O(n \log n)$? Потому что большинство вершин (около $\frac{n}{2}$) находятся внизу (листья) и поднимаются на максимальную высоту $\log n$: $\sum(\frac{n}{2}) \cdot \log n \in \Theta(n \log n)$. В методе `siftDown` всё наоборот: бóльшая часть вершин имеет высоту 0 и 1, а максимальный путь $\log n$ проходит только корень!

—

ТЕМА 17. Пирамидальная сортировка (HeapSort)

Лучшее время	Среднее время	Худшее время	Доп. память
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ **in-place**

1. На какой структуре данных реализуется: Массив с организацией в Max-Heap.

2. Ограничения и свойства:
- **Устойчивость**: НЕТ (перенос последнего элемента массива в корень кучи нарушает порядок одинаковых элементов).
- **In-place**: ДА (память $O(1)$, не требует дополнительных массивов).
- Постоянство времени: гарантированно $\Theta(n \log n)$ во всех случаях.

3. Логика работы:
1. **Фаза 1**: строим Max-Heap на исходном массиве $A[0..n-1]$ за $O(n)$ методом `siftDown`.
2. **Фаза 2**: для $i = n-1$ вниз до 1:
   - Меняем местами корень $A[0]$ (текущий максимум) с $A[i]$;
   - Теперь максимум встал на своё постоянное место в конец массива;
   - Считаем, что размер кучи уменьшился на 1;
   - Запускаем `siftDown(0)` в подмассиве длины i, восстанавливая кучу за $O(\log i)$.
3. Итоговое время: $O(n) + \sum_{i=1}^{n-1} O(\log i) = O(n \log n)$.

> 💡 **Мнемоника для запоминания**
> HeapSort — это оптимизированная сортировка выбором: вместо линейного поиска максимума за $O(n)$ куча отдаёт максимум за $O(1)$ и восстанавливается за $O(\log n)$.

—

ТЕМА 18. Удаление элемента из пирамиды

Лучшее время	Среднее время	Худшее время	Доп. память
Extract-Max: $O(\log n)$	**Удаление по индексу:** $O(\log n)$	**Поиск элемента:** $O(n)$	**Память:** $O(1)$

1. Извлечение корня (Extract-Max / Extract-Min):
1. Сохраняем значение корня: `max_val = A[0]`.
2. Переносим последний элемент массива в корень: $A[0] = A[n-1]$.
3. Уменьшаем размер кучи: $n = n - 1$.
4. Восстанавливаем свойство кучи: вызываем `siftDown(0)`.
5. Время работы: строго $O(\log n)$.

2. Удаление произвольного элемента по индексу i:
1. Заменяем удаляемый элемент последним элементом кучи: $A[i] = A[n-1]$.
2. Уменьшаем размер кучи: $n = n - 1$.
3. Смотрим, как изменилось значение:
   - Если $A[i] > A[\text{parent}(i)]$, то нарушено свойство с предком → вызываем `siftUp(i)`.
   - Если $A[i] <$ детей, то нарушено свойство с потомками → вызываем `siftDown(i)`.
4. Сработает ровно одна из операций. Время: $O(\log n)$.

—

ТЕМА 19. Очередь с приоритетом (Priority Queue)

Лучшее время	Среднее время	Худшее время	Доп. память
Insert: $O(\log n)$	**GetMax:** $O(1)$	**ExtractMax:** $O(\log n)$	**ChangeKey:** $O(\log n)$ **(с хэш-таблицей)**

1. На какой структуре данных реализуется: Двоичная куча + внешний массив / хеш-таблица индексов (`"pos"[id]`).

2. Операции изменения приоритета:
- `increaseKey(i, ""new_p"riority")` (для Max-Heap): присваиваем $A[i] = \text{new_p}$, вызываем `siftUp(i)` ($O(\log n)$).
- `decreaseKey(i, ""new_p"riority")`: присваиваем $A[i] = \text{new_p}$, вызываем `siftDown(i)` ($O(\log n)$).

3. Проблема адресации и решение: Куча не умеет искать элемент по его ID быстрее чем за $O(n)$. Чтобы `decreaseKey` работал за $O(\log n)$ (например, в алгоритме Дейкстры), поддерживают массив обратных позиций: `pos[id] = index`. При любом обмене элементов в куче `swap(A[i], A[j])` синхронно обновляются позиции: `swap(pos[A[i].id], pos[A[j].id])`.

4. Применимость: Алгоритм Дейкстры кратчайших путей ($O((V+E)\log V)$), алгоритм Прима поиска остовного дерева, кодирование Хаффмана.

—

ТЕМА 20. Динамическое программирование (Основы + 3 задачи)

Лучшее время	Среднее время	Худшее время	Доп. память
Полиномиальное время	**Принцип оптимальности**	**Мемоизация / Табуляция**	**Оптимизация памяти**

1. Общая идея ДП:
- **Оптимальная подструктура**: оптимальное решение задачи строится из оптимальных решений её подзадач.
- **Перекрывающиеся подзадачи**: одни и те же подзадачи решаются многократно (в отличие от Divide and Conquer, где подзадачи независимы).

2. Задача 20.1: Кузнечик (Прыжки с монетками):
- Кузнечик прыгает вперед на $1, 2, ..., k$ кочек. Монетки на кочках: $c_1, ..., c_n$.
- **Состояние**: $\text{dp}[i]$ — макс. сумма на пути от кочки 1 до кочки i.
- **База**: $\text{dp}[1] = c_1$.
- **Переход**: $\text{dp}[i] = c_i + \max_{1 \le j \le \min(k, i-1)} \text{dp}[i-j]$.

- **Восстановление пути**: `prev[i] = argmax(dp[i-j])`. Раскрутка с n до 1.
- **Сложность**: Время $O(nk)$ наивно, $O(n)$ с монотонным деком/очередью с максимумом. Память $O(n)$ ($O(k)$ без восстановления пути).

3. Задача 20.2: Рюкзак 0-1 (0-1 Knapsack Problem):
- n предметов с весами w_i и стоимостями c_i, вместимость W. Каждый предмет берется 0 или 1 раз.
- **Состояние**: $\mathrm{dp}[i][w]$ — макс. стоимость предметов среди первых i при весе $\leq w$.
- **База**: $\mathrm{dp}[0][w] = 0$ для всех $w \geq 0$.
- **Переход**:

$$\mathrm{dp}[i][w] = \begin{cases} \mathrm{dp}[i-1][w] & \text{если } w < w_i \\ \max(\mathrm{dp}[i-1][w], \mathrm{dp}[i-1][w-w_i] + c_i) & \text{если } w \geq w_i \end{cases}$$

- **Оптимизация памяти до $O(W)$**: используем один массив $\mathrm{dp}[w]$. Проход по весу w выполняется **СТРОГО СПРАВА НАЛЕВО (от W вниз до w_i)**! Это предотвращает многократный учет одного и того же предмета.
- **Сложность**: Время $O(nW)$ (псевдополиномиальное), Память $O(nW) \to O(W)$.

4. Задача 20.3: Наибольшая возрастающая подпоследовательность (НВП / LIS):
- В массиве $A[0..n-1]$ найти самую длинную строго возрастающую подпоследовательность.
- **Версия $O(n^2)$**: $\mathrm{dp}[i]$ — длина НВП с концом в $A[i]$. $\mathrm{dp}[i] = 1 + \max_{j<i, A[j]<A[i]} \mathrm{dp}[j]$.
- **Версия $O(n \log n)$**: массив $d[\mathrm{len}]$ — минимальный хвост возрастающей подпоследовательности длины len.
  - Массив d строго монотонно возрастает $\to$ применяем бинарный поиск `lower_bound`!
  - Для каждого $x = A[i]$ находим первое $d[j] \geq x$ за $O(\log n)$ и обновляем $d[j] = x$. Если x больше всех, увеличиваем длину: $d[\mathrm{len}++] = x$.

РАЗДЕЛ 3. ПРОДВИНУТЫЕ ТЕМЫ СО ЗВЁЗДОЧКОЙ (*)

ТЕМА *1. Квикселект (Quickselect и алгоритм BFPRT)

Лучшее время	Среднее время	Худшее время	Доп. память
Среднее: $O(n)$	**Худшее:** $O(n)$ **(BFPRT)**	**Худшее:** $O(n^2)$ **(Рандом)**	**Память:** $O(1)$ **in-place**

1. Постановка задачи: Найти k-ю порядковую статистику (элемент, который встал бы на позицию k при полной сортировке массива) без сортировки всего массива.

2. Рандомизированный Quickselect:
- Выбираем случайный опорный элемент pivot, делаем partition.
- Если pivot встал на позицию k, возвращаем его.
- Если $k < \text{pos}$, рекурсивно ищем только в левой части. Иначе — в правой.
- Матожидание времени: $O(n)$. Худший случай: $O(n^2)$ при неудачных разбиениях.

3. Детерминированный Quickselect (BFPRT / Медиана медиан): Алгоритм Блюма, Флойда, Пратта, Ривеста и Тарьяна гарантирует $O(n)$ в худшем случае!
- **Шаг 1**: Разбиваем n элементов на группы по 5 элементов. Всего $\lceil \frac{n}{5} \rceil$ групп.
- **Шаг 2**: В каждой 5-элементной группе находим медиану (сортировкой за $O(1)$, не более 7 сравнений).
- **Шаг 3**: Рекурсивно вызываем Quickselect для поиска медианы полученных $\lceil \frac{n}{5} \rceil$ медиан. Назовем её M (медиана медиан).
- **Шаг 4**: Делаем partition всего массива вокруг M.
- **Шаг 5**: Рекурсивно переходим в одну из половин.

> 📐 **Доказательство O(n) для алгоритма BFPRT**
> Половина из $\frac{n}{5}$ групп имеет свою медиану $\leq M$. В каждой такой группе хотя бы 3 элемента $\leq$ медианы группы, а значит, и $\leq M$. Исключая группу самой медианы медиан и возможную неполную группу, число элементов $\leq M$ гарантированно составляет не менее:
>
> $$3 \cdot \left(\frac{1}{2} \cdot \frac{n}{5} - 2 \right) \geq \frac{3n}{10} - 6.$$
>
> Следовательно, размер подмассива, в который рекурсивно переходит алгоритм на шаге 5, не превышает:
>
> $$n - \left(\frac{3n}{10} - 6 \right) = \frac{7n}{10} + 6.$$
>
> Получаем рекуррентное соотношение для худшего случая:
>
> $$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil \right) + T\left(\frac{7n}{10} + 6 \right) + O(n).$$
>
> Ключевой факт: $\frac{1}{5} + \frac{7}{10} = \frac{2}{10} + \frac{7}{10} = \frac{9}{10} < 1$! По индукции $T(n) \leq cn$:
>
> $$T(n) \leq \frac{c}{5}n + \frac{7c}{10}n + c_1 n = \frac{9c}{10}n + c_1 n \leq cn \quad \text{при} \quad c \geq 10c_1.$$
>
> Таким образом, $T(n) \in O(n)$ в худшем случае!

—

ТЕМА *2. Мастер-теорема как способ доказательства

Лучшее время	Среднее время	Худшее время	Доп. память
Дерево рекурсии	**3 классических случая**	**Критический** $\log_b a$	**Теорема Акра-Баззи**

1. Канонический вид рекуррентного соотношения:

$$T(n) = aT\left(\frac{n}{b} \right) + f(n)$$

где $a \geq 1$ — число подзадач, $b > 1$ — фактор уменьшения размера задачи, $f(n)$ — стоимость разделения и слияния.

2. Дерево рекурсии и критический показатель:
- На уровне i находится a^i узлов размера $\frac{n}{b^i}$.
- Число листьев дерева на глубине $\log_b n$ равно $a^{\log_b n} = n^{\log_b a}$.

- **Критический показатель**: $c_{\text{crit}} = \log_b a$. Он определяет стоимость нижнего уровня (листьев).

3. Три случая мастер-теоремы:

Случай	Соотношение f(n) и листьев	Итоговая оценка T(n)	Пример
1. Листья	$f(n) = O(n^c), c < \log_b a$	$\Theta\left(n^{\log_b a}\right)$	Штрассен, Карацуба
2. Баланс	$f(n) = \Theta\left(n^{\log_b a} \log^k n\right), k \geq 0$	$\Theta\left(n^{\log_b a} \log^{k+1} n\right)$	MergeSort ($k = 0$)
3. Корень	$f(n) = \Omega(n^c), c > \log_b a$ + регулярность	$\Theta(f(n))$	Быстрый сплит

4. Важные классические примеры:
- **MergeSort**: $T(n) = 2T(\frac{n}{2}) + \Theta(n)$. $\log_2 2 = 1$. $f(n) = n^1 \log^0 n \to$ Случай 2 ($k = 0$) $\to \Theta(n \log n)$.
- **Бинарный поиск**: $T(n) = T(\frac{n}{2}) + \Theta(1)$. $\log_2 1 = 0$. Случай 2 ($k = 0$) $\to \Theta(\log n)$.
- **Умножение Карацубы**: $T(n) = 3T(\frac{n}{2}) + \Theta(n)$. $\log_2 3 \approx 1.585 > 1 \to$ Случай 1 $\to \Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585})$.
- **Умножение Штрассена**: $T(n) = 7T(\frac{n}{2}) + \Theta(n^2)$. $\log_2 7 \approx 2.807 > 2 \to$ Случай 1 $\to \Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807})$.

—

ТЕМА *3. Дин прог: битовые маски, профиль, редакционное расстояние

Лучшее время	Среднее время	Худшее время	Доп. память
Битовые маски: $O(n^2 2^n)$	**Профиль:** $O(NM2^N)$	**Левенштейн:** $O(nm)$	**Память:** $\frac{O(2^N)}{O(\min(n,m))}$

1. ДП по битовым маскам (Bitmask DP):
- Подмножество элементов $\{0, ..., N-1\}$ кодируется целым числом $0 \leq \text{mask} < 2^N$.
- Проверка i-го бита: `(mask >> i) & 1`. Установка: `mask | (1 << i)`. Сброс: `mask & ~(1 << i)`.
- **Применение: Задача коммивояжера (TSP)**: $\text{dp}[\text{mask}][i]$ — минимальная стоимость пути, посетившего города из множества mask и завершившегося в городе i. Время $O(n^2 2^n)$ вместо $O(n!)$.

2. Динамика по профилю (Profile DP / Broken Profile):
- Классическая задача: замостить доску $N \times M$ доминошками 2×1. N мало ($N \leq 16$).
- **Состояние**: $\text{dp}[i][\text{mask}]$ — число способов замостить первые $i-1$ столбцов так, чтобы в i-й столбец выступали клетки согласно битовой маске mask.
- **Оптимизация (поклеточная динамика / изломанный профиль)**: обрабатываем доску клетка за клеткой. Контур (профиль) состоит из N соседних ребер. Время перехода $O(1)$, общее время $O(N \cdot M \cdot 2^N)$ вместо экспоненты от площади!

3. Редакционное расстояние (Расстояние Левенштейна / Wagner-Fischer):
- Минимальное число операций (вставка, удаление, замена символа) для превращения строки S_1 в S_2.
- **Состояние**: $\text{dp}[i][j]$ — редакционное расстояние между префиксами $S_1[0..i-1]$ и $S_2[0..j-1]$.
- **База**: $\text{dp}[i][0] = i$ (удаление всех i символов), $\text{dp}[0][j] = j$ (вставка j символов).
- **Переход**:

$$\text{dp}[i][j] = \min \begin{cases} \text{dp}[i-1][j] + 1 & (\text{удаление из } S_1) \\ \text{dp}[i][j-1] + 1 & (\text{вставка в } S_1) \\ \text{dp}[i-1][j-1] + [S_1[i-1] \neq S_2[j-1]] & (\text{совпадение или замена}) \end{cases}$$

- **Сложность**: Время $O(nm)$, Память $O(nm) \to O(\min(n, m))$ скользящим слоем из 2 строк.

—

ТЕМА *4. Дек и Очередь с поддержкой минимума

Лучшее время	Среднее время	Худшее время	Доп. память
Дек: все операции $O(1)$	**Стек с мин:** $O(1)$ **худшее**	**Очередь с мин:** $O(1)$ **аморт.**	**Доп. память:** $O(n)$

1. Дек (Deque - Double-Ended Queue):
- Поддерживает добавление и извлечение с **обоих концов**: `push_front`, `push_back`, `pop_front`, `pop_back` за $O(1)$.
- Реализация: кольцевой буфер на динамическом массиве или массив фиксированных чанков (`std::deque`).

2. Стек с поддержкой минимума за строгое $O(1)$: Хранит пары `(value, min_so_far)`.
- `push(x)`: `min_val = min(x, stack.empty() ? x : stack.top().second); stack.push({x, min_val});`
- `pop()`: удаляет верхнюю пару.
- `getMin()`: `return stack.top().second;` — время $O(1)$ в худшем случае!

3. Очередь с минимумом на двух стеках (S_{in} и S_{out}):

- `enqueue(x)`: `s_in.push(x)` за $O(1)$.
- `dequeue()`: если S_{out} пуст, перекладываем **все** элементы из S_{in} в S_{out} (порядок разворачивается в правильный FIFO!). Затем извлекаем из S_{out}.
- `getMin()`:
  - Если оба стека не пусты: $\min(S_{\text{in}}.\text{getMin}(), S_{\text{out}}.\text{getMin}())$.
  - Иначе: минимум непустого стека. Время строго $O(1)$!
- **Амортизационная сложность**: каждый элемент входит в S_{in} 1 раз, переносится в S_{out} 1 раз и удаляется 1 раз. Итого 3 операции на элемент $\to O(1)$ амортизированно!

4. Применение: Поиск минимума в скользящем окне длины k за линейное время $O(n)$ (вместо $O(n \log k)$ через `std::set`).

—

ТЕМА *5. Бинпоиск: lower_bound, по ответу, вещественный

Лучшее время	Среднее время	Худшее время	Доп. память
Целочисленный: $O(\log n)$	**По ответу:** $O(\log(R) \cdot T_{\text{check}})$	**Вещественный: фикс. итерации**	**Память:** $O(1)$

1. Инвариант полуинтервала $[L, R)$ и `lower_bound`:
- Диапазон поиска задается полуинтервалом $[L, R)$.
- **Безопасное вычисление середины**: `m = L + (R - L) / 2` (предотвращает 32-битное переполнение выражения `(L + R) / 2`).
- `lower_bound(x)`: возвращает наименьший индекс i, где $A[i] \geq x$.

```
int L = 0, R = n;
while (L < R) {
    int m = L + (R - L) / 2;
    if (A[m] >= x) R = m; // ответ в левой половине, включая m
    else L = m + 1;       // ответ строго правее m
}
return L; // индекс первого элемента >= x
```

2. Бинпоиск по ответу (Binary Search on Answer):
- Применяется, когда функция валидности $\text{check}(X)$ монотонна (например, $0, 0, ..., 0, 1, 1, ..., 1$).
- Диапазон возможных ответов $[L, R]$ делится пополам, на каждом шаге вызывается проверка $\text{check}(m)$ за $O(T_{\text{check}})$.
- Общая сложность: $O(\log(R - L) \cdot T_{\text{check}})$.

3. Вещественный бинпоиск:
- Поиск аргумента монотонной непрерывной функции $f(x) = 0$ на $[L, R]$.
- **Золотое правило**: вместо условия `while (R - L > EPS)` (опасного бесконечным циклом из-за машинного эпсилона IEEE 754) использовать **фиксированное число итераций**:

```
for (int iter = 0; iter < 100; ++iter) {
    double m = L + (R - L) / 2.0;
    if (f(m) <= 0) L = m;
    else R = m;
}
```

За 100 итераций исходный отрезок уменьшается в $2^{100} \approx 10^{30}$ раз, что с запасом превосходит точность типа `double` (10^{-16}).